Problema 1

```
import math
import matplotlib.pyplot as plt

def solver_gauss_jordan_premium(A,
b):  n = len(A)
 M = [A[i] + [b[i]] for i in
range(n)]
 for i in range(n):
 max_el = abs(M[i][i])
 max_row = i
 for k in range(i + 1, n):
 if abs(M[k][i]) > max_el:
 max_el = abs(M[k][i])
 max_row = k
 M[i], M[max_row] = M[max_row], M[i]
 piv = M[i][i]
 if abs(piv) < 1e-15:
 continue

 for j in range(i, n + 1):
 M[i][j] /= piv

 for k in range(n):
 if k != i:
 fator = M[k][i]
 for j in range(i, n + 1):
M[k][j] -= fator * M[i][j]

 return [linha[n] for linha in M]
```

```
def gerar_warren(num_triangulos):
 nx, ny = [], []
 for i in range(num_triangulos +
1):  nx.append(float(i * 2))
 ny.append(0.0)
 if i < num_triangulos:
 nx.append(float(i * 2 + 1))
 ny.append(2.0)

 barras = []
 nos = len(nx)
 for i in range(nos - 2):
 barras.append((i, i+1))
 barras.append((i, i+2))
 barras.append((nos-2, nos-1))

 return nx, ny, barras

n_triangulos = 6
nx, ny, barras =
gerar_warren(n_triangulos) N, B =
len(nx), len(barras)

num_incognitas = B + 3
A = [[0.0] * num_incognitas for _ in range(2 * N)]
b = [0.0] * (2 * N)

no_carga = (N // 2)
b[2 * no_carga + 1] = 1000.0
```

```
for i, (u, v) in enumerate(barras):
 dx = nx[v] - nx[u]
 dy = ny[v] - ny[u]
 L = math.sqrt(dx**2 + dy**2)
 c, s = dx/L, dy/L
 A[2*u][i] += c
 A[2*u+1][i] += s
 A[2*v][i] -= c
 A[2*v+1][i] -= s

A[2*0][B] = 1.0
A[2*0+1][B+1] = 1.0
A[2*(N-1)][B+2] = 1.0

 resultado = solver_gauss_jordan_premium(A,
 b) forcas = resultado[:B]

 fig = plt.figure(figsize=(14, 10))
 gs = fig.add_gridspec(2, 2)

 ax1 = fig.add_subplot(gs[0, 0])
 for u, v in barras:
ax1.plot([nx[u], nx[v]], [ny[u], ny[v]], 'ko-', markersize=4, lw=1)
 ax1.set_title("1. Geometria e Nós")

 ax2 = fig.add_subplot(gs[0, 1])
 max_f = max([abs(f) for f in forcas]) if forcas else 1
 for i, (u, v) in enumerate(barras):
  cor = 'blue' if forcas[i] > 0.05 else 'red' if forcas[i] < -0.05 else
```

```
'gray'  espessura = 1 + (abs(forcas[i]) / max_f) * 4
 ax2.plot([nx[u], nx[v]], [ny[u], ny[v]], color=cor,
lw=espessura) ax2.set_title("2. Mapa de Calor (Espessura =
Intensidade)")

ax3 = fig.add_subplot(gs[1, 0])
ax3.bar(range(B), forcas, color=['blue' if f > 0 else 'red' for f in
forcas]) ax3.set_title("3. Esforços por Barra (N)")

ax4 = fig.add_subplot(gs[1, 1])
ax4.hist(forcas, bins=10, color='purple',
edgecolor='white') ax4.set_title("4. Distribuição das
Forças")

plt.tight_layout()
plt.show()
```

Gemini:

Codigo fonte sem edição

```
import math
import matplotlib.pyplot as
plt import time

def gauss_jordan(A,
b):  n = len(A)
 M = []
 for i in range(n):
 linha = A[i][:] + [b[i]]
 M.append(linha)
 for i in range(n):
 piv = M[i][i]
```

```
if abs(piv) < 1e-12:
for k in range(i + 1, n):
if abs(M[k][i]) > 1e-12:
M[i], M[k] = M[k], M[i]
piv = M[i][i]
break
if abs(piv) < 1e-12:
raise ValueError("Sistema singular ou mal condicionado")
for j in range(i, n + 1):
M[i][j] /= piv
for k in range(n):
if k != i:
fator = M[k][i]
for j in range(i, n + 1):
M[k][j] -= faktor * M[i][j]
x = []
for i in range(n):
x.append(M[i][n])
return x

x = [0, 2, 4, 6]
y = [0, 2, 0, 2]
N = len(x)
barras = [(0, 1), (1, 2), (2, 3), (0, 2),
(1, 3)] plt.show()
```

Com edição:

```
import math
import matplotlib.pyplot as plt

def gauss_jordan_manual(A,
b):  n = len(A)
 M = [A[i] + [b[i]] for i in
range(n)]  for i in range(n):
 max_el = abs(M[i][i])
 max_row = i
 for k in range(i + 1, n):
 if abs(M[k][i]) > max_el:
 max_el = abs(M[k][i]); max_row = k
M[i], M[max_row] = M[max_row], M[i]
```

```
piv = M[i][i]
 for j in range(i, n + 1): M[i][j] /= piv
 for k in range(n):
 if k != i:
 fator = M[k][i]
 for j in range(i, n + 1): M[k][j] -= fator * M[i][j]
return [linha[n] for linha in M]


nx = [0, 2, 4, 6]; ny = [0, 2, 0, 2]
barras = [(0,1), (1,2), (2,3), (0,2), (1,3)]
B = len(barras); N = len(nx)
A = [[0.0]*(B+3) for _ in range(2*N)]
b = [0.0]*(2*N)
b[2*1+1] = 1000.0 # Carga no nó 1 em Y


for i, (u, v) in enumerate(barras):
 L = math.sqrt((nx[v]-nx[u])**2 + (ny[v]-ny[u])**2)
c, s = (nx[v]-nx[u])/L, (ny[v]-ny[u])/L
 A[2*u][i], A[2*u+1][i] = c, s
 A[2*v][i], A[2*v+1][i] = -c, -s


A[2*0][B], A[2*0+1][B+1], A[2*2+1][B+2] = 1, 1, 1
sol = gauss_jordan_manual(A, b)
forcas = sol[:B]


fig, axs = plt.subplots(2, 2, figsize=(10, 8))
for u, v in barras: axs[0,0].plot([nx[u], nx[v]], [ny[u], ny[v]],
'k-') for i, (u, v) in enumerate(barras):
 cor = 'red' if forcas[i] < 0 else 'blue'
 axs[0,1].plot([nx[u], nx[v]], [ny[u], ny[v]], color=cor,
```

```
lw=2) axs[1,0].bar(range(B), [abs(f) for f in forcas])

axs[1,1].hist(forcas)

plt.show()
```

Chat gpt:

```
import math

import matplotlib.pyplot as plt


def gauss_jordan(A, b):

 n = len(A)

 M = [A[i] + [b[i]] for i in range(n)]

 for i in range(n):

 piv = M[i][i]

 for j in range(i, n + 1): M[i][j] /= piv
 for k in range(n):

 if k != i:

 fator = M[k][i]

 for j in range(i, n + 1): M[k][j] -= fator * M[i][j]

 return [linha[n] for linha in M]


x = [0, 2, 4, 6]

y = [0, 2, 0, 2]

N = len(x)

barras = [(0, 1), (1, 2), (2, 3), (0, 2), (1, 3)]
```

```
print("Aviso: Este código não possui a lógica de montagem da matriz
de  equilíbrio.")

plt.show()
```

Com edição:

```
import math

import matplotlib.pyplot as plt


def gauss_jordan_manual(A, b):

 n = len(A)

 M = [A[i] + [b[i]] for i in range(n)]

 for i in range(n):

 max_el = abs(M[i][i])
 max_row = i

 for k in range(i + 1, n):

 if abs(M[k][i]) > max_el:

 max_el = abs(M[k][i])

 max_row = k

 M[i], M[max_row] = M[max_row], M[i]

piv = M[i][i]

 for j in range(i, n + 1): M[i][j] /= piv

 for k in range(n):

 if k != i:

 fator = M[k][i]

 for j in range(i, n + 1): M[k][j] -= fator * M[i][j]
```

```
return [linha[n] for linha in M]


nx = [0.0, 2.0, 4.0, 6.0]

ny = [0.0, 2.0, 0.0, 2.0]

barras = [(0,1), (1,2), (2,3), (0,2), (1,3)]

B = len(barras)

N = len(nx)


A = [[0.0] * (B + 3) for _ in range(2 * N)]

b = [0.0] * (2 * N)

b[2*1+1] = 1000.0


for i, (u, v) in enumerate(barras):

 dx = nx[v] - nx[u]

 dy = ny[v] - ny[u]

 L = math.sqrt(dx**2 + dy**2)
c, s = dx/L, dy/L

 A[2*u][i] = c

 A[2*u+1][i] = s

 A[2*v][i] = -c

 A[2*v+1][i] = -s


A[2*0][B] = 1.0

A[2*0+1][B+1] = 1.0

A[2*2+1][B+2] = 1.0


solucao = gauss_jordan_manual(A, b)

forcas = solucao[:B]


fig, axs = plt.subplots(2, 2, figsize=(12, 10))
```

```
for u, v in barras:
 axs[0, 0].plot([nx[u], nx[v]], [ny[u], ny[v]], 'ko-')
axs[0, 0].set_title("Geometria Original")

for i, (u, v) in enumerate(barras):
 cor = 'blue' if forcas[i] > 0.01 else 'red'
 axs[0, 1].plot([nx[u], nx[v]], [ny[u], ny[v]], color=cor,
lw=2) axs[0, 1].set_title("Mapa de Calor (T/C)")

axs[1, 0].bar(range(B), [abs(f) for f in forcas],
color='gray') axs[1, 0].set_title("Magnitude das
Forcas")

axs[1, 1].hist(forcas, bins=5, edgecolor='black')
axs[1, 1].set_title("Histograma")

plt.tight_layout()
plt.show()
```

Claude:

```
import math
import matplotlib.pyplot as plt


def solver(A, b):
 n = len(A)
 for i in range(n):
 piv = A[i][i]
 for j in range(i, n):
 A[i][j] /= piv
 b[i] /= piv
 for k in range(n):
 if k != i:
 f = A[k][i]
 for j in range(i, n):
 A[k][j] -= f * A[i][j]
 b[k] -= f * b[i]
```

```
 return b


def calcular_forcas():
 pass


def plotar_graficos():
 plt.plot([0, 1], [0, 1])
 plt.show()


 print("Erro: Funções definidas mas não integradas com os dados da


treliça.") Com edição:


 import math
 import matplotlib.pyplot as plt


def gauss_jordan_manual(A, b):
 n = len(A)
 M = [A[i] + [b[i]] for i in range(n)]
 for i in range(n):
 max_row = i
 for k in range(i + 1, n):
 if abs(M[k][i]) > abs(M[max_row][i]): max_row = k
M[i], M[max_row] = M[max_row], M[i]  piv = M[i][i]
 if abs(piv) > 1e-12:
 for j in range(i, n + 1): M[i][j] /= piv
 for k in range(n):
 if k != i:
 f = M[k][i]
 for j in range(i, n + 1): M[k][j] -= f * M[i][j]
```

```
return [linha[n] for linha in M]


num_baias = 15
nx, ny = [], []
for i in range(num_baias + 1):
 nx.append(i * 2.0)
 ny.append(0.0)
 nx.append(i * 2.0 + 1.0)
 ny.append(2.0)

barras = []
num_nos = len(nx)
for i in range(num_nos - 2):
 barras.append((i, i+1))
 barras.append((i, i+2))

B = len(barras)
N = len(nx)
A = [[0.0] * (B + 3) for _ in range(2 * N)]
b = [0.0] * (2 * N)
b[2*(N//2)+1] = -1000.0

for i, (u, v) in enumerate(barras):
 L = math.sqrt((nx[v]-nx[u])**2 + (ny[v]-ny[u])**2)
 c, s = (nx[v]-nx[u])/L, (ny[v]-ny[u])/L
 A[2*u][i] += c
 A[2*u+1][i] += s
 A[2*v][i] += -c
 A[2*v+1][i] += -s

A[2*0][B], A[2*0+1][B+1], A[2*(N-1)+1][B+2] = 1.0, 1.0, 1.0
```

```
sol = gauss_jordan_manual(A, b)

forcas = sol[:B]


fig, ax = plt.subplots(2, 1, figsize=(15, 8))

for i, (u, v) in enumerate(barras):

 cor = 'blue' if forcas[i] > 0.1 else 'red'

 ax[0].plot([nx[u], nx[v]], [ny[u], ny[v]], color=cor, lw=1)

ax[0].set_title(f"Treliça Warren Automática (N={N}) - Azul: Tração,
Vermelho:  Compressão")


ax[1].bar(range(B), forcas, color='green')

ax[1].set_title("Esforços em cada Barra")


plt.tight_layout()
plt.show()
```